

Week-01 HandsOn Solution

❖ Advanced SQL Server

File 1: Advanced Concepts

Exercise-1: Ranking and Window Functions

- Code:

//Database Creation:

```
CREATE DATABASE OnlineRetailStore;
GO

USE OnlineRetailStore;
GO

CREATE TABLE Products (
    ProductID INT IDENTITY(1,1) PRIMARY KEY,
    ProductName NVARCHAR(100) NOT NULL,
    Category NVARCHAR(50) NOT NULL,
    Price DECIMAL(10,2) NOT NULL,
    Brand NVARCHAR(50) NOT NULL,
    StockQuantity INT DEFAULT 0,
    CreatedDate DATETIME2 DEFAULT GETDATE()
);
GO

INSERT INTO Products (ProductName, Category, Price, Brand, StockQuantity) VALUES
-- Electronics Category
('iPhone 15 Pro', 'Electronics', 1199.99, 'Apple', 50),
('Samsung Galaxy S24', 'Electronics', 999.99, 'Samsung', 75),
('MacBook Pro 16"', 'Electronics', 2499.99, 'Apple', 25),
('Dell XPS 13', 'Electronics', 1299.99, 'Dell', 30),

-- Clothing Category
('Nike Air Max', 'Clothing', 129.99, 'Nike', 100),
('Levi's 501 Jeans', 'Clothing', 89.99, 'Levi's', 150),
('Adidas Ultraboost', 'Clothing', 180.00, 'Adidas', 80),
('North Face Jacket', 'Clothing', 249.99, 'North Face', 45),

-- Home & Garden Category
('Dyson V15 Vacuum', 'Home & Garden', 749.99, 'Dyson', 35),
('KitchenAid Mixer', 'Home & Garden', 399.99, 'KitchenAid', 25),
('Instant Pot Pro', 'Home & Garden', 149.99, 'Instant Pot', 60),
('Roomba i7+', 'Home & Garden', 599.99, 'iRobot', 15),

-- Sports Category
('Peloton Bike', 'Sports', 1445.00, 'Peloton', 10),
('Bowflex Dumbbells', 'Sports', 349.99, 'Bowflex', 25),
('Yoga Mat Premium', 'Sports', 79.99, 'Manduka', 150),
('Resistance Bands Set', 'Sports', 29.99, 'Fit Simplify', 200);
GO

SELECT 'Database and table created successfully!' AS Status;

SELECT
    'Total Products: ' + CAST(COUNT(*) AS VARCHAR(10)) AS Summary
FROM Products;
```

```

SELECT
    Category,
    COUNT(*) AS ProductCount,
    MIN(Price) AS MinPrice,
    MAX(Price) AS MaxPrice
FROM Products
GROUP BY Category
ORDER BY Category;

```

//Query:

```

USE OnlineRetailStore;
GO

SELECT 'ROW_NUMBER() - Assigns unique sequential ranks' AS RankingType;

SELECT
    ProductID,
    ProductName,
    Category,
    Price,
    Brand,
    ROW_NUMBER() OVER (PARTITION BY Category ORDER BY Price DESC) AS RowNum
FROM Products
ORDER BY Category, Price DESC;
GO

SELECT 'RANK() - Leaves gaps after tied ranks' AS RankingType;

SELECT
    ProductID,
    ProductName,
    Category,
    Price,
    Brand,
    RANK() OVER (PARTITION BY Category ORDER BY Price DESC) AS RankNum
FROM Products
ORDER BY Category, Price DESC;
GO

SELECT 'DENSE_RANK() - No gaps after tied ranks' AS RankingType;

SELECT
    ProductID,
    ProductName,
    Category,
    Price,
    Brand,
    DENSE_RANK() OVER (PARTITION BY Category ORDER BY Price DESC) AS DenseRankNum
FROM Products
ORDER BY Category, Price DESC;
GO

SELECT 'COMPARISON: All Three Ranking Functions' AS ComparisonType;

SELECT
    ProductID,
    ProductName,
    Category,
    Price,
    Brand,
    ROW_NUMBER() OVER (PARTITION BY Category ORDER BY Price DESC) AS RowNum,
    RANK() OVER (PARTITION BY Category ORDER BY Price DESC) AS RankNum,
    DENSE_RANK() OVER (PARTITION BY Category ORDER BY Price DESC) AS DenseRankNum
FROM Products
ORDER BY Category, Price DESC;
GO

SELECT 'TOP 3 Products per Category using ROW_NUMBER()' AS ResultType;

WITH RankedProducts AS (

```

```

SELECT
    ProductID,
    ProductName,
    Category,
    Price,
    Brand,
    StockQuantity,
    ROW_NUMBER() OVER (PARTITION BY Category ORDER BY Price DESC) AS RowNum
FROM Products
)
SELECT
    ProductID,
    ProductName,
    Category,
    Price,
    Brand,
    StockQuantity,
    RowNum
FROM RankedProducts
WHERE RowNum <= 3
ORDER BY Category, RowNum;
GO

SELECT 'TOP 3 Products per Category using RANK()' AS ResultType;

WITH RankedProducts AS (
    SELECT
        ProductID,
        ProductName,
        Category,
        Price,
        Brand,
        StockQuantity,
        RANK() OVER (PARTITION BY Category ORDER BY Price DESC) AS RankNum
    FROM Products
)
SELECT
    ProductID,
    ProductName,
    Category,
    Price,
    Brand,
    StockQuantity,
    RankNum
FROM RankedProducts
WHERE RankNum <= 3
ORDER BY Category, RankNum;
GO

SELECT 'TOP 3 Products per Category using DENSE_RANK()' AS ResultType;

WITH RankedProducts AS (
    SELECT
        ProductID,
        ProductName,
        Category,
        Price,
        Brand,
        StockQuantity,
        DENSE_RANK() OVER (PARTITION BY Category ORDER BY Price DESC) AS DenseRankNum
    FROM Products
)
SELECT
    ProductID,
    ProductName,
    Category,
    Price,
    Brand,
    StockQuantity,
    DenseRankNum
FROM RankedProducts

```

```
WHERE DenseRankNum <= 3
ORDER BY Category, DenseRankNum;
GO
```

• Output:

	ResultType						
1	TOP 3 Products per Category using ROW_NUMBER()						
	ProductID	ProductName	Category	Price	Brand	StockQuantity	RowNum
1	8	North Face Jacket	Clothing	249.99	North Face	45	1
2	7	Adidas Ultraboost	Clothing	180.00	Adidas	80	2
3	5	Nike Air Max	Clothing	129.99	Nike	100	3
4	3	MacBook Pro 16"	Electro...	2499...	Apple	25	1
5	4	Dell XPS 13	Electro...	1299...	Dell	30	2
6	1	iPhone 15 Pro	Electro...	1199...	Apple	50	3
7	9	Dyson V15 Vacu...	Home ...	749.99	Dyson	35	1
8	12	Roomba i7+	Home ...	599.99	iRobot	15	2
9	10	KitchenAid Mixer	Home ...	399.99	KitchenAid	25	3
10	13	Peloton Bike	Sports	1445...	Peloton	10	1
11	14	Bowflex Dumbb...	Sports	349.99	Bowflex	25	2
12	15	Yoga Mat Premi...	Sports	79.99	Manduka	150	3

	ResultType						
1	TOP 3 Products per Category using RANK()						
	ProductID	ProductName	Category	Price	Brand	StockQuantity	RankNum
2	7	Adidas Ultraboost	Clothing	180.00	Adidas	80	2
3	5	Nike Air Max	Clothing	129.99	Nike	100	3
4	3	MacBook Pro 16"	Electro...	2499...	Apple	25	1
5	4	Dell XPS 13	Electro...	1299...	Dell	30	2
6	1	iPhone 15 Pro	Electro...	1199...	Apple	50	3
7	9	Dyson V15 Vacu...	Home ...	749.99	Dyson	35	1
8	12	Roomba i7+	Home ...	599.99	iRobot	15	2
9	10	KitchenAid Mixer	Home ...	399.99	KitchenAid	25	3
10	13	Peloton Bike	Sports	1445...	Peloton	10	1
11	14	Bowflex Dumbb...	Sports	349.99	Bowflex	25	2
12	15	Yoga Mat Premi...	Sports	79.99	Manduka	150	3

	ResultType						
1	TOP 3 Products per Category using DENSE_RANK()						
	ProductID	ProductName	Category	Price	Brand	StockQuantity	DenseRankNum
1	8	North Face Jacket	Clothing	249.99	North Face	45	1
2	7	Adidas Ultraboost	Clothing	180.00	Adidas	80	2
3	5	Nike Air Max	Clothing	129.99	Nike	100	3
4	3	MacBook Pro 16"	Electro...	2499...	Apple	25	1
5	4	Dell XPS 13	Electro...	1299...	Dell	30	2
6	1	iPhone 15 Pro	Electro...	1199...	Apple	50	3
7	9	Dyson V15 Vacu...	Home ...	749.99	Dyson	35	1
8	12	Roomba i7+	Home ...	599.99	iRobot	15	2
9	10	KitchenAid Mixer	Home ...	399.99	KitchenAid	25	3
10	13	Peloton Bike	Sports	1445...	Peloton	10	1
11	14	Bowflex Dumbbb...	Sports	349.99	Bowflex	25	2
12	15	Yoga Mat Premi	Sports	79.99	Manduka	150	3

RankingType						
1	ROW_NUMBER() - Assigns unique sequential ranks					
	ProductID	ProductName	Category	Price	Brand	RowNum
1	8	North Face Jacket	Clothing	249.99	North Face	1
2	7	Adidas Ultraboost	Clothing	180.00	Adidas	2
3	5	Nike Air Max	Clothing	129.99	Nike	3
4	6	Levi's 501 Jeans	Clothing	89.99	Levis	4
5	3	MacBook Pro 16"	Electro...	2499...	Apple	1
6	4	Dell XPS 13	Electro...	1299...	Dell	2
7	1	iPhone 15 Pro	Electro...	1199...	Apple	3
8	2	Samsung Galax...	Electro...	999.99	Samsung	4
9	9	Dyson V15 Vacu...	Home ...	749.99	Dyson	1
10	12	Roomba i7+	Home ...	599.99	iRobot	2
11	10	KitchenAid Mixer	Home ...	399.99	KitchenAid	3
12	11	Instant Pot Pro	Home ...	149.99	Instant Pot	4
13	13	Peloton Bike	Sports	1445...	Peloton	1
14	14	Bowflex Dumbb...	Sports	349.99	Bowflex	2
15	15	Yoga Mat Premi...	Sports	79.99	Manduka	3
16	16	Resistance Ban...	Sports	29.99	Fit Simplify	4

RankingType						
1	RANK() - Leaves gaps after tied ranks					
	ProductID	ProductName	Category	Price	Brand	RankNum
1	8	North Face Jacket	Clothing	249.99	North Face	1
2	7	Adidas Ultraboost	Clothing	180.00	Adidas	2
3	5	Nike Air Max	Clothing	129.99	Nike	3
4	6	Levi's 501 Jeans	Clothing	89.99	Levis	4
5	3	MacBook Pro 16"	Electro...	2499...	Apple	1
6	4	Dell XPS 13	Electro...	1299...	Dell	2
7	1	iPhone 15 Pro	Electro...	1199...	Apple	3
8	2	Samsung Galax...	Electro...	999.99	Samsung	4
9	9	Dyson V15 Vacu...	Home ...	749.99	Dyson	1
10	12	Roomba i7+	Home ...	599.99	iRobot	2
11	10	KitchenAid Mixer	Home ...	399.99	KitchenAid	3
12	11	Instant Pot Pro	Home ...	149.99	Instant Pot	4
13	13	Peloton Bike	Sports	1445...	Peloton	1
14	14	Bowflex Dumbb...	Sports	349.99	Bowflex	2
15	15	Yoga Mat Premi...	Sports	79.99	Manduka	3
16	16	Resistance Ban...	Sports	29.99	Fit Simplify	4

RankingType						
1	DENSE_RANK() - No gaps after tied ranks					
	ProductID	ProductName	Category	Price	Brand	DenseRankNum
1	8	North Face Jacket	Clothing	249.99	North Face	1
2	7	Adidas Ultraboost	Clothing	180.00	Adidas	2
3	5	Nike Air Max	Clothing	129.99	Nike	3
4	6	Levi's 501 Jeans	Clothing	89.99	Levis	4
5	3	MacBook Pro 16"	Electro...	2499...	Apple	1
6	4	Dell XPS 13	Electro...	1299...	Dell	2
7	1	iPhone 15 Pro	Electro...	1199...	Apple	3
8	2	Samsung Galax...	Electro...	999.99	Samsung	4
9	9	Dyson V15 Vacu...	Home ...	749.99	Dyson	1
10	12	Roomba i7+	Home ...	599.99	iRobot	2
11	10	KitchenAid Mixer	Home ...	399.99	KitchenAid	3
12	11	Instant Pot Pro	Home ...	149.99	Instant Pot	4
13	13	Peloton Bike	Sports	1445...	Peloton	1
14	14	Bowflex Dumbb...	Sports	349.99	Bowflex	2
15	15	Yoga Mat Premi...	Sports	79.99	Manduka	3
16	16	Resistance Ban...	Sports	29.99	Fit Simplify	4

File 4: Stored procedure

//Database Creation:

```
CREATE DATABASE EmployeeManagementSystem;
GO

USE EmployeeManagementSystem;
GO

CREATE TABLE Departments (
    DepartmentID INT PRIMARY KEY,
    DepartmentName VARCHAR(100)
);
GO

CREATE TABLE Employees (
    EmployeeID INT PRIMARY KEY,
    FirstName VARCHAR(50),
    LastName VARCHAR(50),
    DepartmentID INT FOREIGN KEY REFERENCES Departments(DepartmentID),
    Salary DECIMAL(10,2),
    JoinDate DATE
);
GO

INSERT INTO Departments (DepartmentID, DepartmentName) VALUES
(1, 'HR'),
(2, 'Finance'),
(3, 'IT'),
(4, 'Marketing');
GO

INSERT INTO Employees (EmployeeID, FirstName, LastName, DepartmentID, Salary, JoinDate) VALUES
(1, 'John', 'Doe', 1, 5000.00, '2020-01-15'),
(2, 'Jane', 'Smith', 2, 6000.00, '2019-03-22'),
(3, 'Michael', 'Johnson', 3, 7000.00, '2018-07-30'),
(4, 'Emily', 'Davis', 4, 5500.00, '2021-11-05'),
(5, 'Robert', 'Wilson', 1, 4800.00, '2020-05-12'),
(6, 'Sarah', 'Brown', 2, 6200.00, '2019-08-14'),
(7, 'David', 'Miller', 3, 7500.00, '2018-12-01'),
(8, 'Lisa', 'Anderson', 1, 5200.00, '2021-02-28'),
(9, 'Kevin', 'Taylor', 3, 6800.00, '2020-09-15'),
(10, 'Maria', 'Garcia', 4, 5800.00, '2021-06-10');
GO
```

Exercise-01: Create a Stored Procedure

• Code:

```
Use EmployeeManagementSystem;
GO

PRINT 'Creating Exercise 1 stored procedures...';

IF OBJECT_ID('sp_GetEmployeesByDepartment', 'P') IS NOT NULL
    DROP PROCEDURE sp_GetEmployeesByDepartment;
GO

IF OBJECT_ID('sp_InsertEmployee', 'P') IS NOT NULL
    DROP PROCEDURE sp_InsertEmployee;
GO

CREATE PROCEDURE sp_GetEmployeesByDepartment
    @DepartmentID INT
```

```

AS
BEGIN
    SET NOCOUNT ON;

    -- Validate input
    IF @DepartmentID IS NULL
    BEGIN
        RAISERROR('DepartmentID cannot be NULL', 16, 1);
        RETURN;
    END

    -- Check if department exists
    IF NOT EXISTS (SELECT 1 FROM Departments WHERE DepartmentID = @DepartmentID)
    BEGIN
        RAISERROR('Department with ID %d does not exist', 16, 1, @DepartmentID);
        RETURN;
    END

    -- Return employee details
    SELECT
        e.EmployeeID,
        e.FirstName,
        e.LastName,
        e.DepartmentID,
        d.DepartmentName,
        e.Salary,
        e.JoinDate
    FROM Employees e
    INNER JOIN Departments d ON e.DepartmentID = d.DepartmentID
    WHERE e.DepartmentID = @DepartmentID
    ORDER BY e.LastName, e.FirstName;
END;
GO

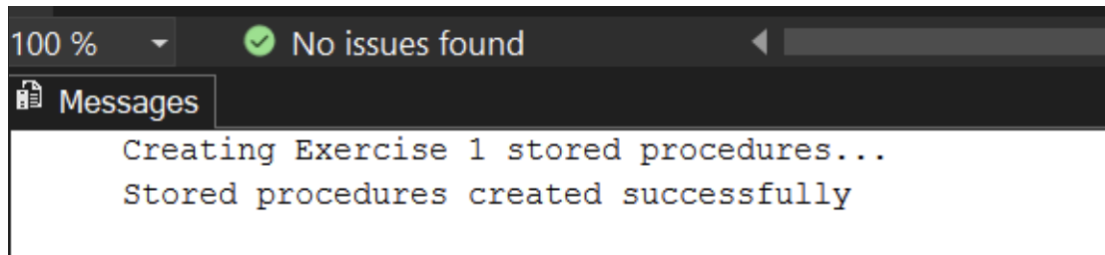
-- Create sp_InsertEmployee
CREATE PROCEDURE sp_InsertEmployee
@FirstName VARCHAR(50),
@LastName VARCHAR(50),
@DepartmentID INT,
@Salary DECIMAL(10,2),
@JoinDate DATE
AS
BEGIN
    -- Validate department exists
    IF NOT EXISTS (SELECT 1 FROM Departments WHERE DepartmentID = @DepartmentID)
    BEGIN
        RAISERROR('Department with ID %d does not exist', 16, 1, @DepartmentID);
        RETURN;
    END

    -- Get next available EmployeeID
    DECLARE @NextEmployeeID INT;
    SELECT @NextEmployeeID = ISNULL(MAX(EmployeeID), 0) + 1 FROM Employees;

    -- Insert new employee
    INSERT INTO Employees (EmployeeID, FirstName, LastName, DepartmentID, Salary, JoinDate)
    VALUES (@NextEmployeeID, @FirstName, @LastName, @DepartmentID, @Salary, @JoinDate);
END;
GO
PRINT 'Stored procedures created successfully';

```

- **Output:**



Exercise 5: Return Data from a Stored Procedure

- **Code:**

```
Use EmployeeManagementSystem;
GO

PRINT 'Creating Exercise 5 Stored Procedure...';

IF OBJECT_ID('sp_GetEmployeeCountByDepartment', 'P') IS NOT NULL
    DROP PROCEDURE sp_GetEmployeeCountByDepartment;
GO

-- Create stored procedure that returns the total number of employees in a department
CREATE PROCEDURE sp_GetEmployeeCountByDepartment
    @DepartmentID INT
AS
BEGIN
    SET NOCOUNT ON;

    -- Validate input parameter
    IF @DepartmentID IS NULL
    BEGIN
        PRINT 'Error: DepartmentID cannot be NULL';
        RETURN;
    END

    -- Check if department exists
    IF NOT EXISTS (SELECT 1 FROM Departments WHERE DepartmentID = @DepartmentID)
    BEGIN
        PRINT 'Error: Department with ID ' + CAST(@DepartmentID AS VARCHAR(10)) + ' does not exist';
        RETURN;
    END

    -- Declare variables for department info
    DECLARE @DepartmentName VARCHAR(100);
    DECLARE @EmployeeCount INT;

    -- Get department name
    SELECT @DepartmentName = DepartmentName
    FROM Departments
    WHERE DepartmentID = @DepartmentID;

    -- Count employees in the specified department
    SELECT @EmployeeCount = COUNT(*)
    FROM Employees
    WHERE DepartmentID = @DepartmentID;

    -- Return the count as a result set
```

```

SELECT
    @DepartmentID AS DepartmentID,
    @DepartmentName AS DepartmentName,
    @EmployeeCount AS TotalEmployees;

-- Also return individual employee details for reference
SELECT
    'Employee Details:' AS Info,
    e.EmployeeID,
    e.FirstName + ' ' + e.LastName AS FullName,
    e.Salary,
    e.JoinDate
FROM Employees e
WHERE e.DepartmentID = @DepartmentID
ORDER BY e.LastName, e.FirstName;
-- Print summary message
PRINT 'Department: ' + @DepartmentName + ' has ' + CAST(@EmployeeCount AS VARCHAR(10)) + ' employees';
END;
GO

PRINT 'Exercise 5 stored procedure created successfully!';

-- TESTING THE STORED PROCEDURES
PRINT '=== TESTING EXERCISE 1 STORED PROCEDURES ===';

-- Test sp_GetEmployeesByDepartment
PRINT 'Testing sp_GetEmployeesByDepartment for HR Department (ID: 1):';
EXEC sp_GetEmployeesByDepartment @DepartmentID = 1;

PRINT ";
PRINT 'Testing sp_GetEmployeesByDepartment for IT Department (ID: 3):';
EXEC sp_GetEmployeesByDepartment @DepartmentID = 3;

PRINT ";
PRINT 'Testing sp_InsertEmployee - Adding a new employee:';
EXEC sp_InsertEmployee
    @FirstName = 'Alice',
    @LastName = 'Johnson',
    @DepartmentID = 2,
    @Salary = 6500.00,
    @JoinDate = '2023-01-15';

PRINT ";
PRINT '=== TESTING EXERCISE 5 STORED PROCEDURE ===';

-- Test sp_GetEmployeeCountByDepartment for each department
PRINT 'Testing sp_GetEmployeeCountByDepartment for HR Department (ID: 1):';
EXEC sp_GetEmployeeCountByDepartment @DepartmentID = 1;

PRINT ";
PRINT 'Testing sp_GetEmployeeCountByDepartment for Finance Department (ID: 2):';
EXEC sp_GetEmployeeCountByDepartment @DepartmentID = 2;

PRINT ";
PRINT 'Testing sp_GetEmployeeCountByDepartment for IT Department (ID: 3):';
EXEC sp_GetEmployeeCountByDepartment @DepartmentID = 3;

PRINT ";
PRINT 'Testing sp_GetEmployeeCountByDepartment for Marketing Department (ID: 4):';
EXEC sp_GetEmployeeCountByDepartment @DepartmentID = 4;

-- ERROR TESTING

PRINT ";
PRINT '=== ERROR TESTING ===';

PRINT 'Testing with invalid department ID (999):';
EXEC sp_GetEmployeesByDepartment @DepartmentID = 999;

PRINT ";
PRINT 'Testing employee count with invalid department ID (999):';

```

```
EXEC sp_GetEmployeeCountByDepartment @DepartmentID = 999;
```

```
-- SUMMARY REPORT
```

```
PRINT ";
```

```
PRINT '=== FINAL SUMMARY ===';
```

```
SELECT 'DEPARTMENT SUMMARY:' AS ReportType;
```

```
SELECT
```

```
    d.DepartmentID,  
    d.DepartmentName,  
    COUNT(e.EmployeeID) AS TotalEmployees,  
    AVG(e.Salary) AS AverageSalary,  
    MIN(e.Salary) AS MinSalary,  
    MAX(e.Salary) AS MaxSalary,  
    SUM(e.Salary) AS TotalSalaryBudget
```

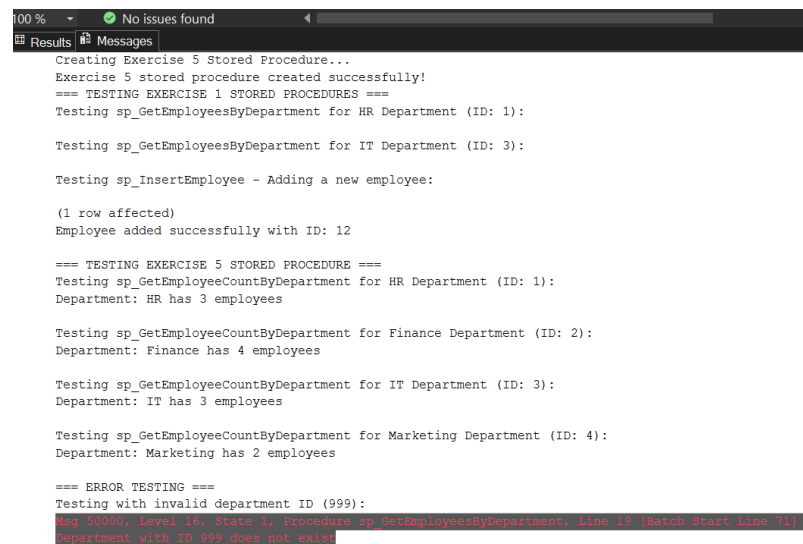
```
FROM Departments d
```

```
LEFT JOIN Employees e ON d.DepartmentID = e.DepartmentID
```

```
GROUP BY d.DepartmentID, d.DepartmentName
```

```
ORDER BY d.DepartmentID;
```

• Output:



```
100 % No issues found  
Results Messages  
Creating Exercise 5 Stored Procedure...  
Exercise 5 stored procedure created successfully!  
=== TESTING EXERCISE 1 STORED PROCEDURES ===  
Testing sp_GetEmployeesByDepartment for HR Department (ID: 1):  
  
Testing sp_GetEmployeesByDepartment for IT Department (ID: 3):  
  
Testing sp_InsertEmployee - Adding a new employee:  
  
(1 row affected)  
Employee added successfully with ID: 12  
  
=== TESTING EXERCISE 5 STORED PROCEDURE ===  
Testing sp_GetEmployeeCountByDepartment for HR Department (ID: 1):  
Department: HR has 3 employees  
  
Testing sp_GetEmployeeCountByDepartment for Finance Department (ID: 2):  
Department: Finance has 4 employees  
  
Testing sp_GetEmployeeCountByDepartment for IT Department (ID: 3):  
Department: IT has 3 employees  
  
Testing sp_GetEmployeeCountByDepartment for Marketing Department (ID: 4):  
Department: Marketing has 2 employees  
  
=== ERROR TESTING ===  
Testing with invalid department ID (999):  
Msg 50000, Level 16, State 1, Procedure sp_GetEmployeesByDepartment, Line 19 [Batch Start Line 71]  
Department with ID 999 does not exist
```

```
Testing employee count with invalid department ID (999):
```

```
Error: Department with ID 999 does not exist
```

```
=== FINAL SUMMARY ===
```

```
(1 row affected)
```

```
(4 rows affected)
```

```
All exercises completed successfully!
```